Criterion C: Development

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY "#")

SETTING UP THE GA 1
SELECTION 2
CROSSOVER 3
MUTATION 5
UPDATING UI 6
PARAMETERS TUNING 7
References 8

SETTING UP THE GA

```
def create_individual(self):
    # First knob (volume) initialized between 1 and 10
    knobs1 = np.random.uniform(1, 10, 1)
    knobs2 = np.random.uniform(1, 10, 1)

    # Other knobs initialized between 0 and 10
    knobs1 = np.append(knobs1, np.random.uniform(0, 10, 2))
    knobs2 = np.append(knobs2, np.random.uniform(0, 10, 2))

    return Individual(knobs1, knobs2)
```

Each individual represents two sound effects devices, each device containing 3 values, each for a knob. From 0 to 10, first knob being the volume. So it is initialised between 1 and 10, as a volume of 0 is undesirable for finding a solution. An individual has a total of 6 values for both effects devices.

```
def evaluate(self, audio, device1, device2):
    wave1 = device1(audio, *self.knobs1)
    wave2 = device2(audio, *self.knobs2)
    self.fitness = np.sqrt(mean_squared_error(wave1, wave2))
    return self.fitness
```

The root mean square error of the two waves represents the fitness. This finds the average difference between the two waves, but the root and the square ensure this value is always positive. The smaller this value is, the better the individual is and the closer the algorithm is to Mr. Rios's theory. This represents the fitness of the individual.

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY "#")

```python
# Track the best individual with the lowest RMSE
if fitness < self.best_fitness:
    self.best_fitness = fitness
    self.best_individual = individual
    self.best_knobs1 = individual.knobs1
    self.best_knobs2 = individual.knobs2

if fitness == 0:
    print("Solution found!")
    print(f"Knobs1 (Device 1): {self.best_knobs1}")
    print(f"Knobs2 (Device 2): {self.best_knobs2}")
    return None  # Terminate the algorithm
```

The best individuals are tracked, providing insight into how the search algorithm is performing. If the RMSE is 0, this also means the two sound waves are identical. Meaning the solution is found and the algorithm has successfully proven my clients theory.

SELECTION

The selection method in a GA is essential for determining which individuals contribute to the next generation; it pushes the algorithm to find the best knob settings where the RMSE is the smallest, attempting to prove Mr Rios's theory. Without selection, the GA would struggle to refine and improve solutions effectively.

```python
def tournament_selection(population, num_individuals, tournament_size):

    selection = [] #individuals that are selected to be in the mating pool

    for _ in range(num_individuals): # iterates as many times as the desired population size

        tournament = random.sample(population, tournament_size) #choses a random sample from
        # the population with a sample size of tournament_size

        best_individual = min(tournament, key=lambda ind: ind.fitness) #the min is chosen from this sample
        # min because we want the lowest RMSE

        selection.append(best_individual) # add that individual to the selection
```

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY "#")

Tournament selection involves randomly choosing a small subset of individuals from the entire population. These individuals then compete in a "tournament," and the winner is selected to join the mating pool (Gridin, 2021). This random selection process encourages diversity within the population. Selection pressure can be controlled by adjusting the tournament size: larger tournaments increase selection pressure, as more individuals compete for a single winner spot, while smaller tournaments reduce this pressure (Brownlee, 2024).

Notably, in tournament selection with a group size of three, the two worst-performing individuals will never be chosen, which can reduce diversity in the population especially with larger tournament sizes. (Gridin, 2021).

```
def roulette_wheel_selection(population, num_individuals):
    selection = [] #individuals that are selected to be in the mating pool

    # Calculate total fitness (using 1 / fitness)
    total_fitness = sum(1 / ind.fitness for ind in population)

    for _ in range(num_individuals): #iterates as many times as the desired population size

        pick = random.uniform(0, total_fitness) #pick is a random number from 0 to the
        #total fitness

        current = 0 #tracks the inividual

        for individual in population: #loops through the whole population

            current += 1 / individual.fitness #adds the fitness of individual

            if current > pick: #when current is larger than the pick...

                selection.append(individual) #add this individual to the selection

                break #once added, current = 0 again and next individual is selected
    return selection
```

In roulette wheel selection, an individual's probability of being chosen for the mating pool is directly proportional to its fitness, with fitter individuals more likely to be selected. This approach encourages convergence toward optimal solutions while maintaining some randomness, which helps preserve diversity within the population. In cases where the goal is to find the individual with the smallest fitness (i.e., the lowest RMSE), the probability of selection is set as the inverse of RMSE (1/RMSE). This way, individuals with lower RMSE values have a higher chance of being selected, aligning the selection pressure with the objective. Roulette wheel logic is inspired by Chrispresso and Gaines (2023).

CROSSOVER

Crossover is a key operator in GAs because it combines genetic material from two parent solutions to create offspring. This process is essential for introducing variation and exploring new

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY "#")

regions of the solution space.

```
def sbx_crossover(parent1, parent2, generation, mutation_rate, eta_c, guassstd):
    # setting up the children
    child1_knobs1 = np.copy(parent1.knobs1)
    child2_knobs1 = np.copy(parent2.knobs1)
    child1_knobs2 = np.copy(parent1.knobs2)
    child2_knobs2 = np.copy(parent2.knobs2)

    for i in range(len(child1_knobs1)): #iterates 3 times
        # Generate a random beta value for each gene
        beta = np.random.uniform(0, 1) #ranhdom float value between 0 and 1 is chosen

        beta_q = (2 * beta)**(1 / (eta_c + 1)) if beta <= 0.5 else (1 / (2 * (1 - beta)))**(1 / (eta_c + 1))
        #beta_q is calculated depending on if beta is larger than or equal to 0.5 or not.

        # Generate child genes
        child1_knobs1[i] = 0.5 * ((1 + beta_q) * parent1.knobs1[i] + (1 - beta_q) * parent2.knobs1[i])
        child2_knobs1[i] = 0.5 * ((1 - beta_q) * parent1.knobs1[i] + (1 + beta_q) * parent2.knobs1[i])

        child1_knobs2[i] = 0.5 * ((1 + beta_q) * parent1.knobs2[i] + (1 - beta_q) * parent2.knobs2[i])
        child2_knobs2[i] = 0.5 * ((1 - beta_q) * parent1.knobs2[i] + (1 + beta_q) * parent2.knobs2[i])

    # Apply Gaussian mutation to both children
    child1_knobs1 = gaussian_mutation(child1_knobs1, mutation_rate, generation, guassstd)[0]
    child2_knobs1 = gaussian_mutation(child2_knobs1, mutation_rate, generation, guassstd)[0]
    child1_knobs2 = gaussian_mutation(child1_knobs2, mutation_rate, generation, guassstd)[0]
    child2_knobs2 = gaussian_mutation(child2_knobs2, mutation_rate, generation, guassstd)[0]

    # Return two new Individuals as children
    child1 = Individual(child1_knobs1, child1_knobs2)
    child2 = Individual(child2_knobs1, child2_knobs2)
    return child1, child2
```

The code is put in equation form below. Where γ is a random generated value.

$$\gamma = (2u)^{\frac{1}{\eta+1}} \quad \text{if } u \leq 0.5$$

$$\gamma = \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta+1}} \quad \text{if } u > 0.5$$

$$\eta = eta_c \text{ (distribution index)}$$

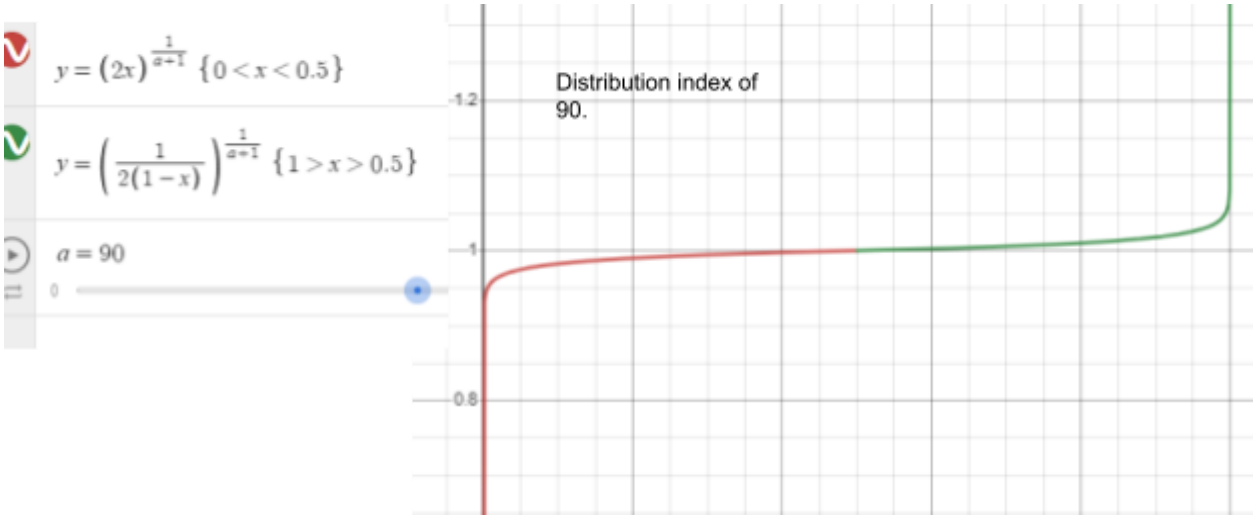
$$child1 = 0.5((1 + \gamma)parent1 + (1 - \gamma)parent2)$$

$$child2 = 0.5((1 - \gamma)parent1 + (1 + \gamma)parent2)$$

The distribution index (η) determines how widely the offspring are spread around the parent solutions. A smaller η results in an exponent further from 0, leading to a greater spread and producing offspring that are more genetically distant from the parent. Conversely, a larger η results in offspring closer to the parent solutions (Eb & Kumar, 1995). The distribution is set to 90, leaning the search towards exploitation, this helps refine solutions, introducing minor variations in the

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY “#”)

children.



(Desmos Graphing Calculator)

MUTATION

Mutation is crucial in GA to maintain diversity and explore new solutions. It introduces random changes, preventing stagnation and helping the algorithm avoid local optima.

```
def gaussian_mutation(knobs, mutation_rate, generation, sigma):
    # Applies Gaussian mutation with a decaying mutation rate.
    # Decay mutation rate as generations increase
    adjusted_mutation_rate = (mutation_rate/np.sqrt(generation+1))

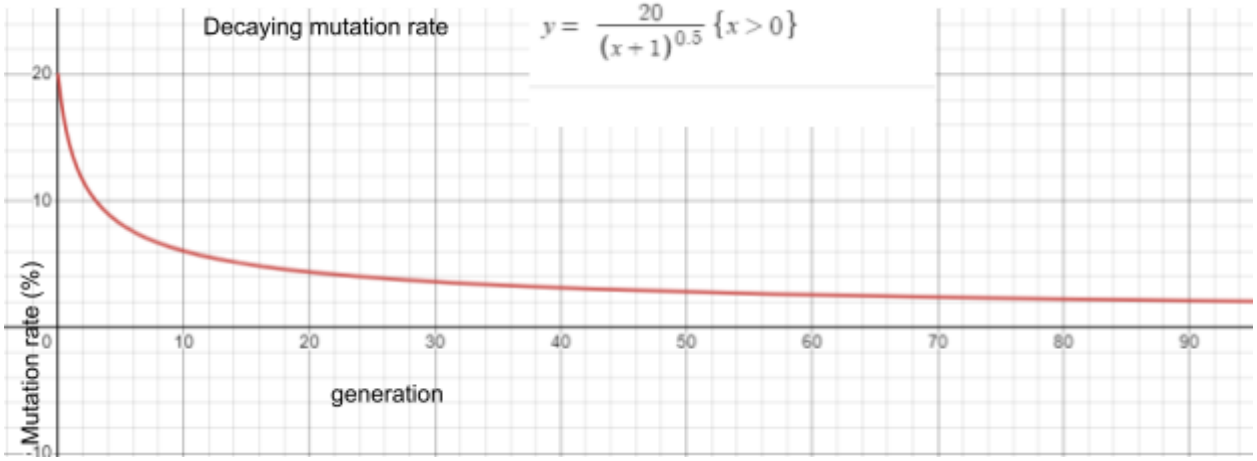
    for i in range(len(knobs)):
        if np.random.rand() < adjusted_mutation_rate:
            # Add Gaussian noise to the knob value
            knobs[i] += np.random.normal(0, sigma)
        # sigma is the standard deviation of the normal distribution curve
        # this values needs to be fined tuned for better performance of the algorithm


    knobs = np.clip(knobs, 0, 10)
    knobs[0] = max(knobs[0], 0.1)# Ensure the volume knob (first knob) stays above 0.1
```

We need to ensure the volume knob is above 0.1 because a volume of 0 of both sound effects devices would result in them sounding identical, but this does not prove Mr. Rios's theory.

Gaussian mutation works by selecting a number from the Gaussian probability distribution, and adding it to the value. A standard deviation is chosen to be 0.037.

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY "#")



(Desmos Graphing Calculator)

Inspired by Luke and Spector (1998), a high mutation rate of 20% is introduced at the start, emphasising exploration and enabling broader search across the solution space. Over time, the mutation rate gradually decreases, allowing for more precise fine-tuning of solutions.

UPDATING UI

```
def step_ga(self):
    avg_fitness, best_fitness, best_knobs1, best_knobs2, avg_knobs1, avg_knobs2 = self.ga.run_generation()
    #retrieve output from the run_generation function

    self.fitness_values.append(avg_fitness) #add the population average fitness to a list
    self.best_fitness_values.append(best_fitness) #adding the best fitness from that generation

    # updating labels
    self.current_generation = self.ga.generation
    self.generation_label.setText(f'Generation: {self.current_generation}')
    self.avg_rmse_label.setText(f'Average RMSE: {avg_fitness:.4f}')
    self.best_rmse_label.setText(f'Smallest RMSE: {best_fitness:.4f}')

    self.plot_generations(self.graphs[2], self.fitness_values, 'g')  # Avg RMSE
    self.plot_generations(self.graphs[3], self.best_fitness_values, 'r')  # Best RMSE

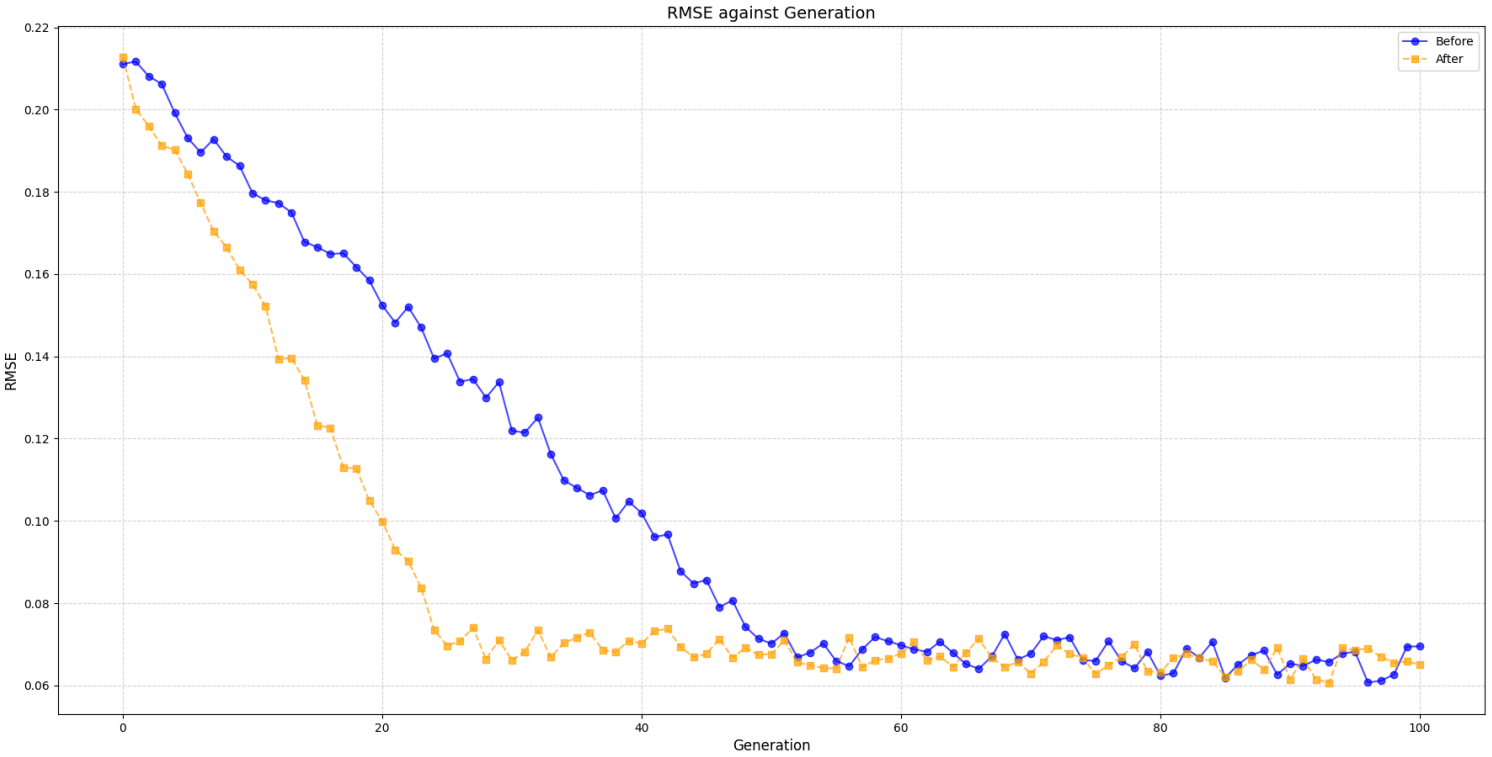
    print(best_knobs1, best_knobs2) #prints the best solutions into the terminal

    if self.current_generation%5==0 or self.current_generation==1: #updates the graph every 5 generations
        self.average_knobs_wave = self.calculate_knobs_wave(avg_knobs1, avg_knobs2) #calculates the nulled waveform
        self.best_knobs_wave = self.calculate_knobs_wave(best_knobs1, best_knobs2)# calculates the best nulled waveform
        self.clear_and_plot(self.graphs[4], self.average_knobs_wave, 'b')  # Avg knobs
        self.clear_and_plot(self.graphs[5], self.best_knobs_wave, 'r')  # Best knobs
```

step_ga function calls the run_generation() function which loops through the processes of 1 cycle of the GA. Moving the algorithm forward by 1 generation, then step_ga retrieves the information and updates the UI, allowing Mr. Rios to get real-time updates and visualisation of the search process.

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY “#”)

PARAMETERS TUNING



A graph showing before and after the parameter tuning process. Meeting the success criteria of improved parameters.

Parameter	Description
population size of 300	Larger population sizes cause longer run times, whereas a smaller population size does not explore the search space enough. A balance has been found to be 300, it performs the best in terms of computation time and performance.
Mating pool size of 100	This encourages the GA to converge, as only one-third of the individuals are chosen to pass on their genetic traits. However, if the mating pool is too small, it may reduce diversity and lead to premature convergence.
Selection method	Roulette Wheel selection method is easy to implement and performs better than the other selection methods. It gives a higher chance to fitter individuals while still leaving a probability for weaker individuals to be selected, maintaining diversity
Mutation parameters	A decaying mutation rate of 20% performed well, introducing more variability. A smaller standard deviation (0.037) has been chosen to

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY “#”)

	balance out the higher mutation rate
Crossover Distribution index of 90	A high distribution index was chosen, as Eb & Kumar (1995) demonstrated that values greater than 80 yielded good results.

word count: 991

References

B., C., & P, M. (1997). Neural networks: A need for caution. *Omega*, *25*(1), 123–133. https://ideas.repec.org/a/eee/jomega/v25y1997i1p123-133.html

Brownlee, J. (2024). *Genetic Algorithm Afternoon: A Practical Guide for Software Developers*. AlgorithmAfternoon.com.

Chrispresso, & Gaines, A. (2023, April 19). *Snake AI*. GitHub. https://github.com/Chrispresso/SnakeAI

Desmos. (n.d.). Desmos graphing calculator. Desmos. Retrieved November 5, 2024, from https://www.desmos.com/calculator

Eb, K., & Kumar, A. (1995). Real-coded Genetic Algorithms with Simulated Binary Crossover: Studies on Multimodal and Multiobjective Problems. *Com Plex Systems*, *9*, 431–454. https://content.wolfram.com/sites/13/2018/02/09-6-1.pdf

Gridin, I. (2021). *Learning Genetic Algorithms with Python*. BPB Publications.

Holland, J. H. (1992). Genetic Algorithms. *Scientific American*, *267*(1), 66–73. https://www.jstor.org/stable/24939139

Luke, S., & Spector, L. (1998). *A Revised Comparison of Crossover and Mutation in Genetic Programming*. https://cs.gmu.edu//~sean/papers/revisedgp98.pdf

Monical, C., & Stonedahl, F. (2014). Static vs. dynamic populations in genetic algorithms for coloring a dynamic graph. *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, 469–476. https://doi.org/10.1145/2576768.2598233

(COMMENTS ARE WRITTEN WITHIN THE CODE, START OF THE COMMENT IS INDICATED BY “#”)